

CHAPTER 3

REQUIREMENT AND ANALYSIS

Requirement Analysis

Requirement analysis defines the users, functions, resources and requirements of the Raksha system. It supports SOS requests, incident and location information, emergency resources, admin management and centralized data storage. It also covers functional, non-functional, hardware, software and database requirements.

3.2 Problem Definition

Problem Statement

Users may face emergencies such as harassment, accidents, medical emergencies, fire, floods and earthquakes where quick and organized information is required. Administrators may also find it difficult to monitor incidents and maintain updated emergency resources.

Main Problems:

1. Lack of a centralized safety platform.
2. Difficulty recording and tracking incidents.
3. Difficulty finding emergency facilities and resources.
4. Lack of organized incident history.
5. Difficulty managing emergency resources.
6. Lack of centralized admin monitoring.
7. Risk of unauthorized access to sensitive data.
8. Difficulty maintaining unsafe-area and risk information.
9. Difficulty generating useful reports and statistics.

3.2.1 Lack of Centralized Safety Records

Emergency incidents may contain important information such as incident type, description, user information, location, priority and current status.

If this information is not stored systematically, it becomes difficult to retrieve, monitor and analyse later.

Raksha provides a centralized database where authorized application components can store and retrieve emergency incident information.

3.2.2 Difficulty in Sharing Location Information

Location helps identify where an incident occurred. Raksha can request the user's location and store available latitude and longitude with the incident. Location accuracy depends on device, GPS, network and permission availability. If unavailable, the system does not assume a false location.

3.2.3 Difficulty in Finding Emergency Resources

During an emergency, users may need quick information about hospitals, shelters, police stations, fire stations and relief centres. Raksha keeps these facility details organized, including name, type, address, contact number, location and operational status. Capacity and operating hours can also be maintained where available. The system separately stores information about registered response teams

3.2.4 Difficulty in Monitoring Emergency Incidents

Administrators require a structured method for monitoring emergency incidents.

Raksha provides an administrative dashboard where authorized administrators can:

- View emergency incidents
- View incident details
- View available location information
- Check incident priority
- Check current status
- Assign registered response resources
- Update incident status
- View incident history
- Monitor system activity

The assignment and status features represent software-based coordination and record management. They do not guarantee physical response from external organizations.

3.2.5 Difficulty in Maintaining Incident History

Emergency incidents may require a chronological record for future reference, reporting and analysis.

Raksha maintains incident updates that record important activities such as:

- Incident creation
- Status change
- Assignment
- Progress update
- Resolution
- Closure
- Cancellation
- Administrative comments

3.2.6 Difficulty in Managing Emergency Resources

Hospitals, shelters, police stations, fire stations, relief centres and response teams may have different information.

Raksha therefore separates emergency resources into structured database records. Facilities such as hospitals and shelters are maintained through the Facilities collection, while registered response teams are maintained through the Rescue Teams collection. Administrators can add, edit, activate or deactivate resource information according to their permissions. The application displays stored

resource information and does not assume real-time availability unless such information is actually connected to and maintained by a reliable source.

4.1.11 Notification Management

Notifications keep users informed about important system events such as incident creation, acknowledgement, assignment, status change, resolution, cancellation, and system updates.

Supported channels may include In-app, Email, SMS, and WhatsApp, depending on configured services.

Notification records store Incident ID, Contact ID, Channel, Status, Provider Response, Attempt Count, Last Attempt Time, and Created Date.

Possible statuses are QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED, and UNAVAILABLE.

3.3 Proposed Solution

Raksha provides a centralized web application that allows registered users to create emergency incidents and access emergency-resource information.

When an SOS request is submitted, the system creates an incident record and stores the available incident details in the database. If location permission is granted, available location information is associated with the incident.

Administrators can monitor incidents through the administrative dashboard, update incident status, manage facilities and response-team information, and maintain administrative records.

The system can also provide notification functionality through configured communication services. Notification delivery depends on the availability and configuration of the selected provider.

The system may use a Python-based risk assessment service to process available incident and location-related factors and provide a risk score and risk level. The risk assessment is intended as software-based decision support and does not replace professional emergency assessment.

System Limitations

Raksha does not claim to provide

- Automatic police FIR registration
- Guaranteed police response
- Guaranteed ambulance response
- Guaranteed rescue-team response
- Exact rescue ETA
- Guaranteed 24×7 emergency response
- Real-time hospital bed availability without a connected source
- Offline emergency communication unless specifically implemented

Actual emergency response depends on authorized external organizations, service providers, network connectivity and real-world availability.

3.4 Requirements Specification

Requirements specification defines the functions and quality requirements of Raksha.

1. **The requirements are divided into:**
2. Functional Requirements
3. Non-Functional Requirements
4. User Requirements
5. System Constraints
6. Hardware Requirements
7. Software Requirements
8. Database Requirements

3.4.1 Functional Requirements

The major functional requirements of Raksha are:

1. User registration and login
2. User authentication
3. Role-based authorization
4. User profile management
5. Emergency contact management
6. SOS and incident creation
7. Incident information management
8. Location processing
9. Emergency facility management
10. Rescue-team management
11. Rescue assignment management
12. Incident status management
13. Notification management
14. Incident history management
15. Unsafe-area reporting
16. Risk-zone management
17. Risk assessment
18. Administrative dashboard
19. Analytics and reporting
20. Resource search and filtering
21. Administrative activity logging
22. Secure access to protected information

3.4.1.1 User Authentication and Authorization

The system shall provide authentication so that only registered users can access protected features.

Requirements:

- The system shall allow users to register.
- The system shall allow users to log in.
- The system shall validate login credentials.
- Passwords shall not be stored in plain text.
- Passwords shall be securely hashed.
- The system shall identify the role of the logged-in account.
- Unauthorized users shall not access administrator functions.
- Users shall be able to log out.
- Protected API operations shall require proper authorization.
- Authentication shall use secure token-based access.

3.4.1.2 Role-Based Access Control

Raksha separates system operations according to user roles.

User

- A normal user can:
- Register
- Login
- Logout
- Manage profile
- Manage emergency contacts
- Create emergency incidents
- Share location where available
- View own incidents
- View incident status
- View notifications
- View emergency resources
- Submit unsafe-area reports
- View relevant risk information where provided

Administrator

An administrator can:

- Login securely
- View emergency incidents
- View incident details
- View available locations
- Update incident status
- Manage facilities
- Manage rescue-team records
- Manage rescue assignments
- Review unsafe-area reports
- Manage risk-zone information

- View dashboard statistics
- View reports
- Monitor administrative activity

3.4.1.3 User Profile Management

The system shall provide basic profile functionality.

Users may:

- View profile information
- Update permitted profile information
- Manage account information
- View emergency history
- Manage emergency contacts
- Logout from the application

Only required information should be collected.

3.4.1.4 Emergency Contact Management

The system shall allow users to maintain emergency contact information.

An emergency contact may contain:

- Contact name
- Phone number
- Email address
- Relationship
- SMS notification preference
- Email notification preference

Emergency contact information shall only be accessible to authorized users and system functions.

3.4.1.5 SOS / Emergency Incident Management

SOS management is one of the main functions of Raksha.

When a user submits an SOS or emergency request, the application creates an incident record.

An incident can contain:

- Incident ID
- User ID
- Incident type
- Category
- Description
- Priority

- Location ID
- Current status
- Creation date and time
- Resolution information, where applicable

Incident Types

Examples include:

- Women Safety
- Medical Emergency
- Accident
- Fire
- Flood
- Earthquake
- Other Emergency

Incident Status

The system uses the following detailed workflow:

REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED

An incident may also be marked:

CANCELLED

where applicable.

In the database design, the SOS or emergency request is stored as an Incident record.

3.4.1.6 Location Processing

The system can request location permission from the user's browser or device.

When available, the system may capture:

- Latitude
- Longitude
- Address or location label
- City
- State
- Country
- Location accuracy
- Location capture time

The system shall:

- Request permission where required.
- Capture coordinates when available.
- Associate location data with the relevant incident.
- Handle unavailable location data.

- Avoid creating false location information.
- Restrict access to sensitive location information.

Location accuracy depends on the user's device, browser permissions, GPS/network conditions and configured location services.

3.4.1.7 Emergency Facility Management

Raksha maintains information about emergency facilities.

The Facilities collection may contain:

- Facility name
- Facility type
- Location ID
- Phone number
- Capacity, where maintained
- Operational status
- Operating hours
- Created date
- Updated date

Facility types may include:

- Hospital
- Shelter
- Police Station
- Fire Station
- Relief Centre

The system does not claim real-time bed availability or actual operational response unless such information is connected to a reliable source.

3.4.1.8 Rescue Team Management

The system can maintain records of registered response teams.

A rescue-team record may contain:

- Team name
- Team type
- Contact information
- Email
- Active status
- Service area
- Specialization

Team types may include:

- Police
- Medical
- Fire
- Volunteer
- NGO

The system manages these records as software information and does not guarantee real-world team availability.

3.4.1.9 Rescue Assignment Management

Administrators can create a software-based assignment record between an incident and a registered response team.

The assignment may contain:

- Incident ID
- Team ID
- Assigned by
- Assigned date and time
- Assignment status
- Notes

Assignment status may include:

- ASSIGNED
- EN_ROUTE
- ON_SCENE
- COMPLETED
- CANCELLED

These statuses represent information recorded within the application and should not be interpreted as guaranteed physical movement or response.

3.4.1.10 Incident Status and Monitoring

Administrators can monitor emergency incidents through the dashboard.

The dashboard may display:

- Total incidents
- Reported incidents
- Acknowledged incidents
- Assigned incidents
- In-progress incidents
- Resolved incidents
- Closed incidents
- Cancelled incidents
- Recent incidents
- Incident categories

- Priority information

When an administrator changes an incident status, the system stores the change and can create an incident-update record and notification.

3.4.1.11 Notification Management

Notifications help users stay updated about important events in the system. They may be sent when an emergency is created, acknowledged, assigned, updated, resolved, or cancelled.

Depending on available services, notifications can be sent through- In-app notifications- Email, SMS, WhatsApp

The system can store basic details such as incident ID, contact, channel, status, attempts, and date. Notification status may be QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED, or UNAVAILABLE.

3.4.1.12 Incident History

Raksha maintains a chronological activity record for incidents.

The history can contain:

- Incident creation
- Status change
- Assignment
- Progress update
- Resolution
- Closure
- Cancellation
- Administrative comment

The Incident Updates collection is used for maintaining this chronological history.

3.4.1.13 Unsafe Area Reporting

Users can report locations that they consider unsafe or risky.

An unsafe-area report may contain:

- Reporter ID
- Location ID
- Category
- Description
- Severity
- Verification status
- Created date
- Updated date

Administrators can review and verify submitted reports.

A user-submitted report should not automatically be treated as a confirmed dangerous location until it has been reviewed according to the implemented workflow.

3.4.1.14 Risk Zone Management

The system can maintain structured information about identified risk zones.

A risk-zone record may contain:

- Zone name
- Risk level
- Geographic boundary
- Risk factors
- Last assessment date
- Active status

Risk-zone information shall be managed according to administrator permissions.

3.4.1.15 Risk Assessment

Raksha can use a Python and Fast API-based risk assessment service to process available information related to an incident or location.

The service may produce:

- Risk score
- Risk level
- Model version
- Input factors
- Assessment time

The risk score may use a range such as 0–100.

Risk levels may include:

- LOW
- MEDIUM
- HIGH
- CRITICAL

Risk assessment is intended to support information analysis. It does not provide guaranteed predictions or replace professional emergency decisions.

3.4.1.16 Disaster Category Management

The system can maintain structured disaster and emergency categories.

A category may contain:

- Category name

- Category code
- Description
- Default priority
- Response-team requirement
- Active status

This allows incident categories to be maintained without modifying the complete application structure.

3.4.1.17 Administrative Dashboard

The administrative dashboard provides a central management interface.

It can display:

- Total users
- Total incidents
- Reported incidents
- Active incidents
- Resolved incidents
- Cancelled incidents
- Available facilities
- Registered rescue teams
- Recent activities
- Notifications
- Basic statistics

Dashboard information shall be generated from actual stored system data.

3.4.1.18 Analytics and Reporting

The system can generate basic analytics from stored data.

Examples include:

- Number of incidents
- Incidents by type
- Incidents by priority
- Incidents by status
- Monthly incident count
- Number of facilities
- Number of shelters
- Number of registered response teams
- Unsafe-area reports
- Risk-level distribution

Analytics represent stored application data and should not be presented as guaranteed predictions of future emergencies.

3.4.1.19 Resource Search

Users can search and filter emergency resources.

Filtering may be based on:

- Resource type
- Facility name
- Area
- Location
- Service category
- Operational status

Where a configured map service is available, location-based information may also be displayed through a map interface.

3.4.1.20 Administrative Activity Logging

Administrative actions should be recorded for accountability.

An administrative log may contain:

- Administrator ID
- Action
- Target type
- Target ID
- Action details
- IP address, where collected
- Date and time

Administrative logs should be accessible only to authorized administrators.

3.4.1.21 Data Management

The system shall maintain structured records for:

1. Users
2. Emergency Contacts
3. Incidents
4. Locations
5. Notifications
6. Rescue Teams
7. Rescue Assignments
8. Incident Updates
9. Unsafe Area Reports
10. Risk Zones
11. Risk Assessments
12. Facilities
13. Disaster Categories

14. Administrative Logs

15. Reports

Database operations shall be performed through authorized backend APIs.

3.4.2 Non-Functional Requirements

The major non-functional requirements are:

1. Security
2. Performance
3. Usability
4. Responsiveness
5. Reliability
6. Scalability
7. Maintainability
8. Compatibility
9. Data Integrity
10. Availability

3.4.2.1 Security

The system shall protect:

- User accounts
- Passwords
- Emergency incidents
- Location information
- Emergency contacts
- Administrative information
- Risk-related information

JWT-based authorization and secure password hashing shall be used for protected operations.

3.4.2.2 Performance

Common operations should provide reasonable response time under normal project usage, including:

- Login
- SOS submission
- Resource search
- Dashboard loading
- Status update
- Notification creation

3.4.2.3 Usability

The application should be simple and easy to understand.

Important functions should have clear labels such as:

- SOS / Emergency

- Submit Request
- View Status
- Find Hospital
- Find Shelter
- Emergency Contacts
- Admin Dashboard

3.4.2.4 Responsiveness

The application should support:

- Desktop
- Laptop
- Tablet
- Mobile phone

The interface should adjust according to screen size.

3.4.2.5 Reliability

After successful submission, the incident should remain stored in the database.

The system should handle failed external services without corrupting the incident record.

3.4.2.6 Scalability

The system should allow additional users, incidents, resources and administrative records without requiring a complete redesign.

3.4.2.7 Maintainability

The application should be divided into logical modules such as:

- Authentication
- User Management
- SOS/Incident Management
- Location
- Facilities
- Rescue Teams
- Notifications
- Risk Assessment
- Administration
- Analytics

3.4.2.8 Compatibility

The web application should work with commonly used modern browsers.

3.4.2.9 Data Integrity

The system should maintain accurate relationships between database records.

For example, an incident should reference a valid user and an available location record where applicable.

3.4.2.10 Availability

The application should remain available when the hosting environment, database, network and required external services are operational.

3.4.3 User Requirements

Raksha mainly supports two primary system roles:

1. User

The user is the person who uses Raksha to create emergency incidents and access available safety information.

Users shall be able to:

- Register
- Login
- Logout
- Manage profile
- Manage emergency contacts
- Create SOS incidents
- Select incident type
- Enter incident description
- Share location where available
- View incident status
- View incident history
- View notifications
- Search emergency resources
- Submit unsafe-area reports

2. Administrator

The administrator manages system information and monitors emergency incidents.

Administrators shall be able to:

- Login securely
- View incidents
- View incident details
- View available locations
- Update incident status
- Manage facilities
- Manage rescue teams
- Manage assignments
- Review unsafe-area reports
- Manage risk-zone information
- View dashboard statistics
- Generate reports
- Monitor administrative activities

3.4.4 User vs Administrator

Function	User	Administrator
Register	Yes	Usually pre-created
Login	Yes	Yes
Manage Profile	Yes	Limited

Create SOS	Yes	Optional
View Own Incidents	Yes	Yes
View All Incidents	No	Yes
Update Incident Status	No	Yes
Manage Facilities	No	Yes
Manage Rescue Teams	No	Yes
Create Assignments	No	Yes
View Notifications	Yes	Yes
Submit Unsafe Area Report	Yes	Yes
Verify Reports	No	Yes
Manage Risk Zones	No	Yes
View Dashboard	Limited	Yes
Analytics	Limited	Yes
Generate Reports	No/Limited	Yes

3.4.5 System Constraints

The following constraints apply to Raksha:

1. The system requires internet connectivity for normal web-based functionality.
2. Location functionality depends on browser/device permission.
3. Location accuracy depends on the device and available location services.
4. Map functionality depends on the configured map service.
5. External email, SMS and WhatsApp services depend on provider availability and configuration.
6. Database availability affects application functionality.
7. Hosting performance affects response time.
8. Unauthorized users must not access protected information.
9. Location information must be protected.
10. Emergency response cannot be guaranteed by the software.
11. Automatic police FIR generation is not assumed.
12. Exact rescue ETA is not assumed.
13. 24x7 police or ambulance integration is not assumed unless actually implemented.
14. Real-time hospital bed availability is not assumed unless connected to a reliable source.
15. Offline emergency functionality is not assumed unless specifically implemented.
16. Risk assessment results are software-generated decision-support information and are not guaranteed predictions.
17. Notification delivery depends on configured external communication providers.

3.4.6 Hardware Requirements

Raksha does not require specialized hardware.

Component/hardware	Requirement
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent
RAM	Minimum 8 GB
Storage	Minimum 256 GB SSD
Display	1080p or higher recommended
Network	Stable Internet connection
Development Device	Normal laptop or desktop

Mobile Device	Smartphone with location capability recommended
---------------	---

3.4.7 Software Requirements

The final software stack should reflect the technologies actually used in the implemented project.

Component	Technology
Frontend	React.js
Frontend Language	TypeScript
Build Tool	Vite
Styling	Tailwind CSS
Backend Runtime	Node.js
Backend Framework	Express.js
API Architecture	REST API
AI Backend	Python
AI Framework	FastAPI
AI Integration	Configured AI Service API
Database	MongoDB/superbase
ODM	Mongoose
Authentication	JSON Web Token (JWT)
Password Security	bcrypt
Location	Browser/Device Geolocation API
Maps	Configured Map Service
Communication	Configured Email/SMS/WhatsApp Providers
Development Environment	Visual Studio Code, Bolt
Version Control	Git, GitHub
Testing	Jest / Supertest, where implemented
Deployment	Configured Cloud Hosting Environment

3.4.8 Database Requirement and Collection Mapping

Raksha uses MongoDB for storing application data. Mongoose can be used at the application layer to define schemas and validate database records.

MongoDB does not use traditional relational foreign keys in the same way as a relational database. Therefore, relationships between records are maintained through application-level references and validation.

The proposed database contains the following collections:

No.	Collection	Purpose
1	Users	User accounts and profile information
2	EmergencyContacts	User emergency contacts
3	Incidents	SOS and emergency incident records
4	Locations	Geographic location information
5	Notifications	Notification records and delivery status
6	RescueTeams	Registered response-team information

7	RescueAssignments	Incident-to-team assignment records
8	IncidentUpdates	Incident history and status updates
9	UnsafeAreaReports	User-submitted unsafe-area reports
10	RiskZones	Structured risk-zone information
11	RiskAssessments	Risk scores and assessment results
12	Facilities	Hospitals, shelters, police/fire stations and relief centres
13	DisasterCategories	Emergency/disaster categories
14	AdminLogs	Administrative activity records
15	Reports	Generated reports and analytics records

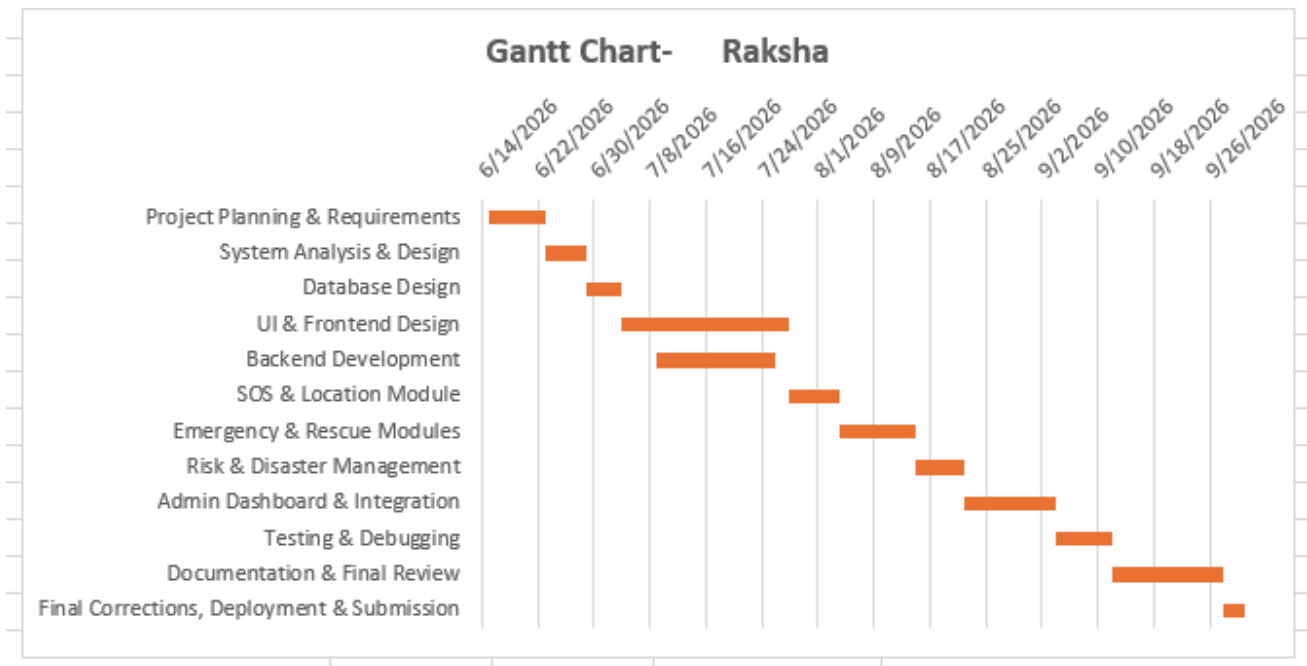
3.5.1 Planning and Scheduling

Sr. No.	Project Activity	Start Date	End Date	Duration	Project Done By
1	Project Planning & Requirements	15-Jun-2026	22-Jun-2026	8 Days	Vivek & Vaibhav – Topic selection, planning and requirements
2	System Analysis & Design	23-Jun-2026	28-Jun-2026	6 Days	Vivek – System flow; Vaibhav – UML and design
3	Database Design	29-Jun-2026	03-Jul-2026	5 Days	Vivek – Database structure; Vaibhav – Collections and fields
4	UI & Frontend Design	04-Jul-2026	27-Jul-2026	24 Days	Vivek – User interface; Vaibhav – Admin interface
5	Backend Development	09-Jul-2026	25-Jul-2026	17 Days	Vivek – APIs; Vaibhav – Backend and database connection
6	SOS & Location Module	28-Jul-2026	03-Aug-2026	7 Days	Vivek – SOS; Vaibhav – Location features
7	Emergency & Rescue Modules	04-Aug-2026	14-Aug-2026	11 Days	Vivek – Resource search; Vaibhav – Rescue team and assignments
8	Risk & Disaster Management	15-Aug-2026	21-Aug-2026	7 Days	Vivek & Vaibhav – Risk zones, assessment and disaster features
9	Admin Dashboard & Integration	22-Aug-2026	03-Sep-2026	13 Days	Vivek – System support; Vaibhav – Dashboard; Both – Integration
10	Testing & Debugging	04-Sep-2026	11-Sep-2026	8 Days	Vivek & Vaibhav – Testing and error fixing
11	Documentation & Final Review	12-Sep-2026	27-Sep-2026	16 Days	Vivek – Requirements; Vaibhav – Design; Both – Final review

12	Final Corrections, Deployment & Submission	28-Sep-2026	30-Sep-2026	3 Days	Vivek & Vaibhav – Final corrections, deployment and submission
----	--	-------------	-------------	--------	--

3.5.1 Gantt Chart

The Gantt Chart shows the planned schedule of the Raksha project, including requirement analysis, system design, development, database implementation, testing, documentation, and final deployment. It helps monitor the progress and completion of each project activity.



3.5.2 PERT CHART

PERT (Program Evaluation and Review Technique) is used to estimate the time required to complete the Raksha – Women Safety and Disaster Management System. It represents the major project activities in a network form and shows the sequence in which the activities are carried out.

Optimistic Time (O): Minimum expected time

Most Likely Time (M): Normal expected time

Pessimistic Time (P): Maximum expected time

Expected Time is calculated as:

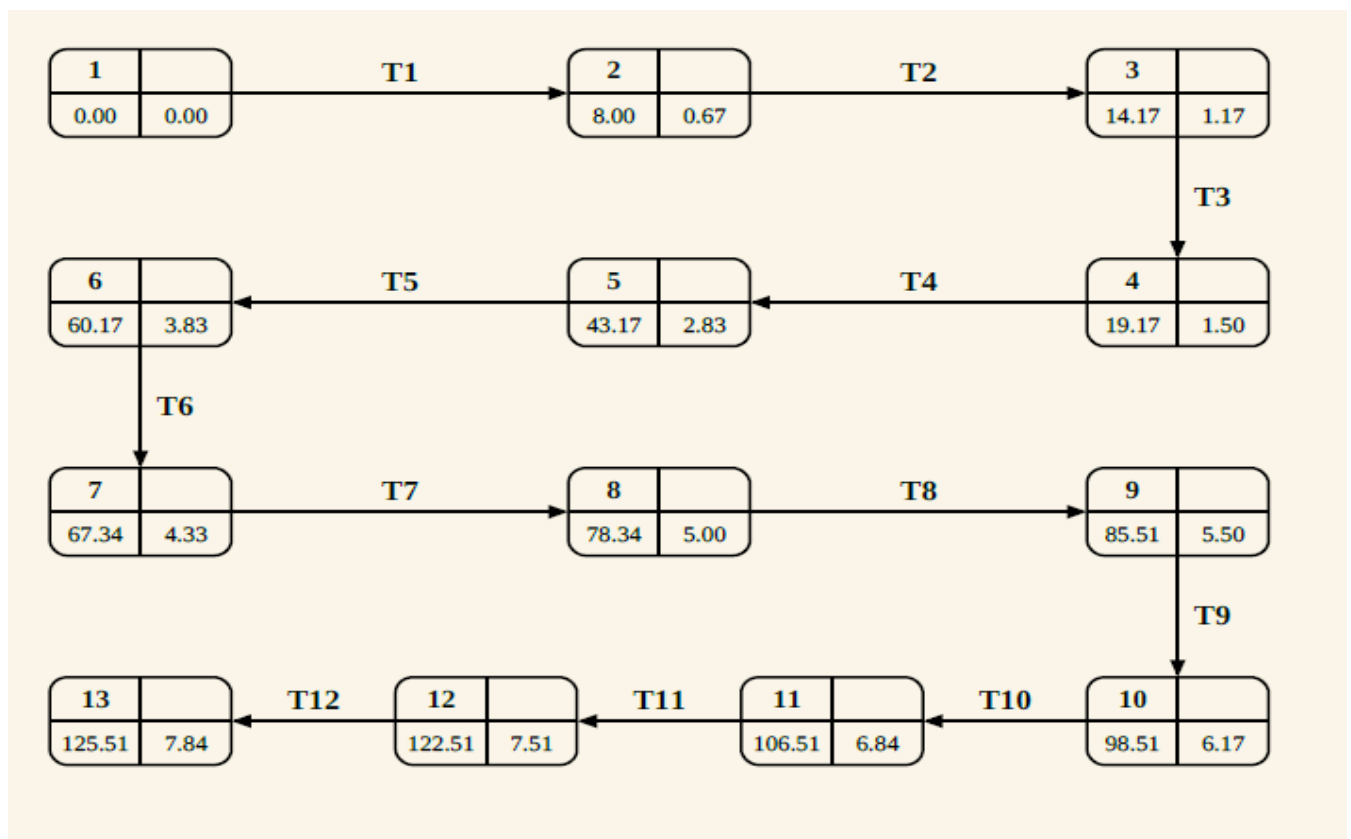
$$TE = (O + 4M + P) / 6$$

Standard Deviation:

$$S.D. = (P - O) / 6$$

PERT Time Estimation Table – RAKSHA

Task	Project Activity	Optimistic Time (O)	Most Likely Time (M)	Pessimistic Time (P)	Expected Time (TE)	S.D.
T1	Project Planning & Requirements	6	8	10	8.00	0.67
T2	System Analysis & Design	5	6	8	6.17	0.50
T3	Database Design	4	5	6	5.00	0.33
T4	UI & Frontend Design	20	24	28	24.00	1.33
T5	Backend Development	14	17	20	17.00	1.00
T6	SOS & Location Module	6	7	9	7.17	0.50
T7	Emergency & Rescue Modules	9	11	13	11.00	0.67
T8	Risk & Disaster Management	6	7	9	7.17	0.50
T9	Admin Dashboard & Integration	11	13	15	13.00	0.67
T10	Testing & Debugging	6	8	10	8.00	0.67
T11	Documentation & Final Review	14	16	18	16.00	0.67
T12	Final Corrections, Deployment & Submission	2	3	4	3.00	0.33



3.6.2 Data Dictionary

The Data Dictionary describes the fields, data types and constraints of the major MongoDB collections used in the Raksha – Women Safety and Disaster Management System. The system contains 15 main collections for managing users, emergency incidents, locations, notifications, rescue resources, risk information, administrative activities and reports.

1. Users

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	Name	String	Required
3	Email	String	Required, Unique
4	phone	String	Required, Unique
5	passwordHash	String	Required, Hashed
6	Role	String	Required, Enum: USER / ADMIN

7	language	String	Optional
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

2. Emergency Contacts

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	userId	ObjectId	Reference → Users
3	Name	String	Required
4	Phone	String	Required
5	Email	String	Optional
6	relationship	String	Optional
7	notifyViaSms	Boolean	Required
8	notifyViaEmail	Boolean	Required
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

3. Incidents

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	userId	ObjectId	Reference → Users
3	Type	String	Required, Safety / Disaster
4	category	String	Required
5	description	String	Required
6	priority	String	Required, LOW / MEDIUM / HIGH / CRITICAL
7	status	String	Required, Enum
8	locationId	ObjectId	Reference → Locations, Optional
9	createdAt	Date	Auto-generated
10	resolvedAt	Date	Optional

4. Locations.

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	latitude	Number	Required
3	longitude	Number	Required
4	address	String	Optional
5	City	String	Optional
6	state	String	Optional
7	country	String	Optional
8	accuracy	Number	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

5. Notifications

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	incidentId	ObjectId	Reference → Incidents
3	contactId	ObjectId	Reference → EmergencyContacts, Optional
4	channel	String	Required, Email / SMS / WhatsApp / In-App
5	Status	String	Required, Enum
6	providerResponse	String	Optional
7	attemptCount	Number	Required
8	lastAttemptAt	Date	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

6. RescueTeams

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required
3	teamType	String	Required, Police / Medical / Fire / NGO / Volunteer
4	phone	String	Required
5	email	String	Optional
6	isActive	Boolean	Required
7	specializations	Array	Optional
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

7. RescueAssignments

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	incidentId	ObjectId	Reference → Incidents
3	teamId	ObjectId	Reference → RescueTeams
4	assignedBy	ObjectId	Reference → Users
5	assignedAt	Date	Auto-generated
6	status	String	Required, Enum
7	notes	String	Optional
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

8. IncidentUpdates

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	incidentId	ObjectId	Reference → Incidents
3	updatedBy	ObjectId	Reference → Users
4	statusFrom	String	Optional

5	statusTo	String	Required
6	comment	String	Optional
7	createdAt	Date	Auto-generated

9. UnsafeAreaReports

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	reportedBy	ObjectId	Reference → Users
3	locationId	ObjectId	Reference → Locations
4	category	String	Required
5	description	String	Required
6	severity	String	Required
7	isVerified	Boolean	Required
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

10. RiskZones

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required
3	riskLevel	String	Required
4	geometry	Object	Required
5	factors	Array	Optional
6	lastAssessedAt	Date	Optional
7	isActive	Boolean	Required
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

11. RiskAssessments

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	locationId	ObjectId	Reference → Locations
3	riskScore	Number	Required, Range: 0–100
4	riskLevel	String	Required, LOW / MEDIUM / HIGH / CRITICAL
5	modelVersion	String	Required
6	inputFactors	Array	Optional
7	assessedAt	Date	Auto-generated
8	createdAt	Date	Auto-generated

12. Facilities

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required
3	facilityType	String	Required, Enum

4	locationId	ObjectId	Reference → Locations
5	phone	String	Required
6	capacity	Number	Optional
7	isOperational	Boolean	Required
8	operatingHours	String	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

13. Disaster Categories

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	Name	String	Required
3	Code	String	Required, Unique
4	description	String	Optional
5	defaultPriority	String	Required
6	requiresResponseTeam	Boolean	Required
7	isActive	Boolean	Required
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

14. AdminLogs

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	adminId	ObjectId	Reference → Users
3	action	String	Required
4	targetType	String	Required
5	targetId	ObjectId	Optional
6	details	String	Optional
7	ipAddress	String	Optional
8	createdAt	Date	Auto-generated

15. Reports

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	generatedBy	ObjectId	Reference → Users
3	Title	String	Required
4	reportType	String	Required
5	filters	Object	Optional
6	dataSnapshot	Object	Optional
7	format	String	Required, PDF / CSV / JSON
8	createdAt	Date	Auto-generated
9	expiresAt	Date	Optional